

INT8 Transformer Attention Accelerator

Engineering Knowledge Transfer — DE2-115 / Cyclone IV Integer-Native Softmax Research

Target hardware	Terasic DE2-115 (Intel/Altera Cyclone IV EP4CE115F29C7)
Repository	github.com/Shailesh-A-hub/int8-transformer-de2-115
Primary clock	50.0 MHz nominal (package pin pending — see §8)
Current milestone	RTL verified cycle-accurate in ModelSim (18/18 tests passed) → entering Quartus synthesis phase
Event	NextGen-Innovathon 2026, Track 2 — Emerging Tech & AI Hardware
Source documents merged	KNOWLEDGE_BASE.md · IMPLEMENTATION_NOTES.md · TEST_PLAN.md · modelsim_transcript.log · PINOUT_REQUIRED.md

0. Read This First

We are building **one** small INT8 Transformer attention accelerator on the DE2-115. The research contribution is **not** "we built a Transformer on an FPGA." It is:

Research goal: Compare three different hardware implementations of the Softmax stage inside the *same* Transformer, and determine the latency / resource / accuracy trade-off between them.

The three implementations — **Version A** (conventional fixed-point detour, baseline), **Tier 0** (integer-native base-2 shift), and **Tier 1** (integer-native + 16-entry LUT refinement) — must share an identical model, weights, input, MAC architecture and clock. Only the Softmax stage changes. That is what makes this a controlled experiment rather than three unrelated demos.

Most important rule: "18/18" means 6 hand-picked demonstration sentences × 3 Softmax variants passed in ModelSim simulation. It is RTL functional verification — **not** a Fluent Speech Commands dataset accuracy result. Do not present it as one.

1. Build Status & Milestone Tracker

Milestone sequence as defined in *IMPLEMENTATION_NOTES.md*, cross-checked against what *modelsim_transcript.log* actually exercises end-to-end.

#	Milestone	Status	Note
1	Restoring divider (24-bit sequential)	■ Done	Corner-case tested — see §5
2	Tier 0 Softmax	■ Done	1,880 Softmax cycles, verified
3	Tier 1 Softmax	■ Done	1,880 Softmax cycles, best numerical accuracy
4	Version A Softmax (baseline)	■ Done	2,008 Softmax cycles, verified
5	Q/K/V MAC datapath	■ Done	8-lane INT8 MAC array

#	Milestone	Status	Note
6	Attention integration	■ Done	attention_engine.sv, 11-stage FSM
7	Classifier	■ Done	Argmax over 6 intent classes, verified in full-system testbench
8	UART telemetry	■ Sim-verified	Packet framing verified in ModelSim; physical PC link pending
9	Quartus resource/timing measurement	■ NEXT	Not started — see §8

Reading down this table: the full RTL engineering phase (1–8) is complete and passes simulation. The project's entire remaining critical path is item 9 — Quartus synthesis — plus the downstream measurements it unlocks (§8, §9).

2. Project Overview & Research Question

2.1 The problem

Transformers rely on floating-point arithmetic inside the Softmax normalization stage. On resource-constrained edge FPGAs with no hard floating-point unit — such as the Cyclone IV EP4CE115 — this typically forces a **conventional fixed-point detour**: INT8 scores → descale → wider fixed-point → 256-entry exponential LUT → sum → divider → requantize → INT8 attention matrix. This detour adds latency (extra pipeline stages), memory footprint (256-entry tables), and multiplier usage for scaling.

2.2 Research question (canonical wording)

Can an integer-native Softmax architecture eliminate the fixed-point detour overhead on a resource-constrained Cyclone IV FPGA, achieving lower latency and hardware resource utilization while preserving classification accuracy?

2.3 Target application workload

Task	Edge spoken intent classification (derived from Fluent Speech Commands)
Model	Single-head INT8 Transformer attention datapath
Sequence length	$L = 8$ tokens
Embedding dimension	$D = 32$
Vocabulary size	32 tokens
Intent classes (6)	turn_on_lights, turn_off_lights, increase_volume, decrease_volume, heat_on, heat_off

What this is not: not a full/ChatGPT-scale Transformer; not an ASR system (input is text/token, not raw audio); not "three separate projects" — one accelerator, three interchangeable Softmax implementations, run as a controlled experiment.

3. The Experiment — Three Softmax Architectures

Everything outside Softmax (embedding lookup, Q/K/V projections, QK^T , $A \times V$, mean pooling, classifier, argmax) is strictly identical across all three experiments.

3.1 Version A — Conventional Fixed-Point Detour (baseline)

1. Find max $m = \max(S)$ (8 cyc) · 2. Descale: $z = S - m$, bounded to $[-8,0]$ (8 cyc) · 3. 256-entry exponential-LUT ROM lookup, accumulate Σw (8 cyc) · 4. Sequential division via 24-bit restoring divider, $w \times 127 \div \Sigma w$ ($8 \times 24 = 192$ cyc) · 5. Requantize to INT8 $[0,127]$ (8 cyc).

Total Softmax latency: 2,008 cycles (251 cycles/row $\times$ 8 rows).

3.2 Tier 0 — Integer-Native Base-2 Shift (proposed)

Exploits $e^z = 2^{(z \cdot \log_2(e))}$. Find max (8 cyc) → linear shift-amount scale, $t = (-z) \times 23$ where $23/16 \approx 1.4375 \approx \log_2(e)$ → power-of-two barrel shift $w = 32768 \llcorner t[7:4]$, accumulate Σw (8 cyc) — **zero exponential-ROM lookups required** → same 24-bit restoring divider ($8 \times 24 = 192$ cyc).

Total Softmax latency: 1,880 cycles (235 cycles/row $\times$ 8 rows) — saves 128 cycles by eliminating the descale and requantize stages.

3.3 Tier 1 — Base-2 Shift + 16-Entry Fractional LUT (proposed)

Refines Tier 0 by splitting t into integer part $q = t[7:4]$ and 4-bit fraction $f = t[3:0]$, then reads a tiny 16-word correction ROM for $2^{(-f)}$:

```
lut[0]=16'hffff; lut[1]=16'hf525; lut[2]=16'heac0; lut[3]=16'he0cc;  
lut[4]=16'hd744; lut[5]=16'hce24; lut[6]=16'hc566; lut[7]=16'hbd08;  
lut[8]=16'hb504; lut[9]=16'had58; lut[10]=16'ha5fe; lut[11]=16'h9ef5;  
lut[12]=16'h9837; lut[13]=16'h91c3; lut[14]=16'h8b95; lut[15]=16'h85aa;  
w = lut[f] ■ q, accumulate  $\Sigma w$ , then normalize via the same 24-bit restoring divider.
```

Total Softmax latency: 1,880 cycles — identical to Tier 0, but with materially better numerical accuracy (§6.2).

4. Hardware Architecture & RTL Implementation

4.1 RTL file hierarchy

```
rtl/
transformer_pkg.sv      # Architectural parameters (L=8, D=32, LANES=8, CLASSES=6)
restoring_divider.v     # Parameterized 24-bit restoring divider FSM
softmax_tier0.sv        # Tier 0 sequential FSM (shift-only, no exp-LUT ROM)
softmax_tier1.sv        # Tier 1 sequential FSM (16-entry LUT + divider)
softmax_detour_vA.sv    # Version A sequential FSM (descale, 256-LUT, requant, divider)
int8_mac_array.v        # 8-lane signed INT8 multiplier-accumulator
attention_engine.sv     # 11-stage master FSM, full dynamic attention pipeline
weight_roms.v          # Parameter storage, pre-loaded via $readmemh from mem/*.hex
cycle_counter.v         # Hardware execution cycle counter, latched output
uart_tx.v, uart_rx.v    # 115,200-baud UART transmitter / receiver
de2_115_top.sv          # Top-level module: board-facing ports + 7-seg decoders
```

4.2 Hardware-first rule — dynamic runtime vs. static storage

A common flaw in trivial FPGA AI demos is precomputing outputs offline. This project deliberately avoids that:

Offline / stored in ROM	Only pre-trained weight matrices (W_Q , W_K , W_V , W_{cls}) and vocabulary embeddings.
On-chip, every inference	Token dispatch → embedding generation → Q/K/V linear projections (8-lane MAC array) → $S = QK^T \blacksquare 15$ → Softmax(S) row-by-row → $H = A \cdot V \blacksquare 7$ → mean pooling over $L=8$ → classifier logits → $\text{argmax}(\text{class} \in [0,5])$.

Team rule: Do not precompute Q/K/V for the actual input sequence offline. Only model weights and static embeddings may be precomputed — this rule must not be broken by anyone editing the RTL.

4.3 The divider bottleneck discovery (a real bug, now fixed)

Initial simulations used Verilog's combinational / operator ($w[j] * 127 / \text{sumw}$), causing all three architectures to execute in an identical, suspicious 3,712 cycles — a red flag, since Version A does strictly more work.

Root cause: ModelSim evaluated division as instantaneous/combinational — not realistic hardware timing. **Fix:** a parameterizable 24-bit sequential restoring divider (`restoring_divider.v`); each division now consumes exactly 24 clock cycles. This exposed the true hardware trade-off in §6.

Why this matters for the pitch: catching a simulator quietly lying about timing, and fixing it with a proper multi-cycle model, is real verification discipline — it makes every other number in this document more credible, not less.

5. Verification Strategy & Coverage

Three-tier verification strategy (per *TEST_PLAN.md*): (1) Python numerical/algorithmic validation, (2) RTL cycle-accurate simulation in ModelSim, (3) Quartus physical/timing verification (§8, not yet run).

5.1 Restoring divider — corner cases tested (*tb/tb_restoring_divider.v*)

Divisor = 1 · powers-of-two divisors · large divisors ($2^{24} - 1$) · dividend < divisor (quotient = 0, remainder = dividend) · exact division (remainder = 0) · pseudo-random 24-bit dividend/divisor pairs. **Status: PASS — 100% quotient and remainder match.**

5.2 Softmax unit tests — Tier 0 & Tier 1 (*tb_softmax_tier0.sv*, *tb_softmax_tier1.sv*)

Monotonicity (larger logits → larger-or-equal weights) · max-element fidelity ($\text{argmax}(\text{logits}) = \text{argmax}(\text{Softmax output})$) · all-equal logits (uniform output $\approx 127/8 = 15$) · large negative dynamic range (bounded exp saturation, no underflow wrap-around) · zero-sum safety (divider handles degenerate zero-sum gracefully) · output range bounding, strictly confined to $[0, 127]$. **Status: PASS — both tiers pass all functional edge cases.**

5.3 Attention engine & full-system integration

Start/done handshaking across all sub-modules · the complete dynamic runtime sequence end to end · Softmax mode selection via `SW[1:0]` · cycle counter latching exact elapsed cycles per run · UART telemetry ASCII packet framing (`I:<intent_id> C:<cycle_count>\r\n`) at 115,200 baud · full regression across 6 intent sentences × 3 Softmax variants = 18 configurations. **Status: 18/18 PASS, 0 errors, 0 warnings.**

5.4 Python numerical validation (*python/softmax_sim.py*)

1,000-vector comparative PTQ suite ($L=8$, logit domain $[-64, +64]$) comparing a 64-bit floating-point reference against Version A / Tier 0 / Tier 1, reporting MAE, maximum error, KL divergence and rank-inversion rate (results in §6.2). Also: a restoring-divider self-check against Python's native `//` and `%` operators across dividends $[1, 32767]$ with divisor 7 — **0 mismatches**; and a deterministic test-vector exporter to `mem/softmax_vectors.txt`.

Reproduce at any time from the repo root:

```
python python/softmax_sim.py --vectors 1000 --length 8
```

Reference output is logged in `python_validation_output.txt`.

6. Current Experimental Results

Simulation executed via `scripts/run_sim_modelsim.ps1` on ModelSim ASE 10.5b. **These are ModelSim RTL-simulation results — not physical board measurements.**

6.1 Cross-architecture timing summary

Variant	Softmax cyc	Total cyc	Latency @50MHz	Vectors passed	Delta
Version A (Detour)	2,008	5,688	113.76 μ s	6/6	baseline
Tier 0 (Shift-Only)	1,880	5,560	111.20 μs	6/6	-128 cyc
Tier 1 (16-LUT Refine)	1,880	5,560	111.20 μs	6/6	-128 cyc

*Note on percentages — two valid conventions appear across project docs, describing the same 128-cycle delta: a **6.37% reduction** relative to Version A's 2,008 cycles (128/2008), equivalently a **1.068 $\times$ / 6.8% speedup ratio** (2008/1880). Both are correct; state which convention you're using when you quote a number.*

6.2 Numerical error metrics (Python PTQ simulation, 1,000 vectors vs. FP64 reference)

Variant	MAE	Max Error	KL Divergence	Rank Inversion %
Version A	0.100967	0.401563	0.532291	0.00%
Tier 0	0.002312	0.009236	0.016769	0.00%
Tier 1	0.000246	0.000979	0.014876	0.00%

Key takeaway: Tier 1 gives $\sim 10\times$ lower MAE than Tier 0 and $\sim 400\times$ lower MAE than Version A, at the identical 1,880-cycle latency as Tier 0 — using only a 16-word ROM.

6.3 ModelSim execution evidence (verbatim excerpt)

Full log: `docs/modelsim_transcript.log`. Key summary block, run 2026-09-06, 0 errors / 0 warnings:

```
=====
CROSS-ARCHITECTURE HARDWARE TIMING & RESOURCE PROFILING SUMMARY
=====
Softmax Variant      | Softmax Cyc | Total Cyc | Latency @50MHz | Accuracy  | Delta
-----
Version A (Detour)   |      2008 c |      5688 c |    113.76 us | 6/6 (100%) | baseline
Tier 0 (Shift-Only)  |      1880 c |      5560 c |    111.20 us | 6/6 (100%) | -128 cycles
Tier 1 (16-LUT Refine) |      1880 c |      5560 c |    111.20 us | 6/6 (100%) | -128 cycles
=====
>>> ALL SYSTEMVERILOG END-TO-END ACCELERATOR INFERENCE TESTS PASSED! <<<
Errors: 0, Warnings: 0
```

Read carefully: The transcript's own summary line reads "Tier 0: 0 LUTs, 0 DSPs (pure barrel shifter)." This is an **architectural** statement — it means the exponentiation datapath needs no ROM table or multiplier. It is **not** a Quartus chip-resource measurement: the FSM and the shared restoring divider still consume real Logic Elements and registers. Do not repeat "0 LUTs, 0 DSPs" as a whole-design resource claim — see §7.

7. Strict Claim Boundaries — Read Before Writing Any Slide

When presenting, writing the report, or answering judges, use only the left column. The right column is explicitly prohibited until the stated measurement exists.

Category	Permitted / defensible	Prohibited / invalid
Latency	"RTL-simulated latency at nominal 50 MHz is 5,560 cycles (111.20 μ s) for T0/T1 vs 5,688 cycles (113.76 μ s) for Version A."	"Measured hardware board latency." (needs physical timer/scope verification)
Accuracy	"6/6 selected test vectors passed (18/18 across the 3 variants)."	"100% FSC benchmark accuracy." (needs full 30,000+ sample dataset evaluation)
Speedup	"Integer-native Softmax achieves 1.068 $\times$ (6.8%) Softmax-stage speedup and 2.25% end-to-end speedup, saving 128 cycles/inference."	"10 $\times$ faster Transformer inference." (the divider dominates execution time)
Division	"Normalization executes sequentially on a 24-bit restoring divider (192 divider cycles/row), zero combinational division."	"Zero-overhead Softmax."
Hardware resources	"Architectural analysis predicts lower LE/memory footprint by removing the 256-entry LUT and scaling multipliers; final numbers pending Quartus post-fit."	Any specific LE, DSP, or BRAM count before Quartus synthesis has actually been run
"0 LUTs, 0 DSPs"	"Tier 0 mathematically eliminates the 256-entry exponential ROM and multipliers from the exponentiation datapath."	Calling it a Quartus resource measurement — FSM, registers, and the divider still consume LEs

8. Quartus Synthesis & Physical Verification Plan

PINOUT_REQUIRED.md warning: Populate quartus/int8_transformer_de2_115.qsf from the official Terasic DE2-115 reference/pin-assignment file or an existing verified Quartus template. Do not invent or assume pin numbers — the RTL top-level intentionally exposes generic board-facing ports.

8.1 Project setup

Target device: **Cyclone IV E EP4CE115F29C7**. Project file: quartus/int8_transformer_de2_115.qpf.
Constraints: quartus/int8_transformer_de2_115.sdc (50 MHz on port CLOCK_50).

8.2 Synthesis checklist

1. Verify pin assignments in the .qsf per the warning above.
2. Run full compilation: Analysis & Synthesis → Fitter (Place & Route) → TimeQuest → Assembler (scripts/run_quartus_compile.ps1, or Ctrl+L in Quartus).
3. Extract from .fit.rpt: total Logic Elements, dedicated registers, embedded multiplier/DSP elements, total memory bits/M9K blocks — for Version A, Tier 0, and Tier 1 separately.
4. Extract from .sta.rpt: worst-case setup slack @ 50.0 MHz, and maximum Fmax across Slow 85°C, Slow 0°C, and Fast 0°C corners.
5. Compute true hardware execution time: $T_{\text{exec}} = \text{Total Cycles} \div \text{Fmax}$ (not cycles $\div$ nominal 50 MHz).

8.3 Physical board demonstration checklist

1. Program the Cyclone IV EP4CE115F29C7 via USB-Blaster (output_files/int8_transformer_de2_115.sof).
2. Select intent sentence via slide switches SW[4:2].
3. Select Softmax variant via slide switches SW[1:0] (00=Tier 0, 01=Tier 1, 10=Version A).
4. Trigger inference via pushbutton KEY[1].
5. Verify winning Intent ID on HEX0 and LEDR[2:0].
6. Verify cycle count on HEX7–HEX2 (expect 5,560 for T0/T1; 5,688 for Version A).
7. Capture telemetry packet on PC via UART (115,200 baud, 8-N-1).

9. DE2-115 Pinout & Live Demonstration Guide

Status: Exact FPGA package pin assignments must come from the official Terasic DE2-115 reference file — not invented. The table below maps RTL ports to board functions; physical pin numbers are a Quartus integration task (§8).

RTL port	Board feature	Dir	Function
CLOCK_50	On-board oscillator	In	50.0 MHz master clock
KEY[0]	Pushbutton 0	In	Active-low async reset
KEY[1]	Pushbutton 1	In	Active-low inference trigger
SW[1:0]	Slide switches 1..0	In	Softmax mode: 00=Tier0, 01=Tier1, 10=Version A
SW[4:2]	Slide switches 4..2	In	Demo sentence select (0–5)
LEDR[2:0]	Red LEDs 2..0	Out	Winning intent ID, binary (0–5)
LEDG[0]	Green LED 0	Out	Accelerator busy
LEDG[1]	Green LED 1	Out	Inference done pulse
HEX0[6:0]	7-seg display 0	Out	Winning intent ID digit
HEX1[6:0]	7-seg display 1	Out	Active Softmax mode (0 / 1 / A)
HEX7–HEX2	7-seg displays 7..2	Out	Latched inference cycle count
UART_TXD	RS-232 / USB-UART	Out	Telemetry @115,200 baud: I:<intent_id> C:<cycle_count>\r\n
UART_RXD	RS-232 / USB-UART	In	Reserved for future command input

9.1 The 4-layer demonstration strategy

Layer 1 — Application demo: switch selects command → KEY[1] triggers inference → HEX0 shows intent. **Layer 2 — Telemetry:** live UART packet to PC showing exact cycles and latency. **Layer 3 — Scientific comparison:** toggle SW[1:0] between Version A and Tier 0 live, show the 7-segment cycle count drop from 5688 to 5560. **Layer 4 — Hardware proof:** present the Quartus LE/DSP/BRAM report once available.

10. Repository Map

Built from actual file paths cited across the source documents — not a guessed template.

```
<repo root>/
  rtl/
    transformer_pkg.v, restoring_divider.v,
    softmax_tier0.v, softmax_tier1.v, softmax_detour_vA.v,
    int8_mac_array.v, attention_engine.v, weight_roms.v,
    cycle_counter.v, uart_tx.v, uart_rx.v, de2_115_top.v
  tb/
    tb_restoring_divider.v, tb_softmax_tier0.v,
    tb_softmax_tier1.v, tb_attention_engine.v,
    tb_full_transformer.v
  python/
  mem/
  quartus/
  scripts/
  output_files/
  docs/
    softmax_sim.py
    softmax_vectors.txt (exported test vectors)
    int8_transformer_de2_115.qpf / .qsf / .sdc
    run_sim_modelsim.ps1, run_quartus_compile.ps1
    int8_transformer_de2_115.sof (after synthesis)
    KNOWLEDGE_BASE.md, IMPLEMENTATION_NOTES.md,
    TEST_PLAN.md, modelsim_transcript.log,
```

11. Teammate Onboarding

11.1 Recommended reading order

1. This document (single consolidated overview)
2. docs/KNOWLEDGE_BASE.md — full architectural and project specification
3. docs/IMPLEMENTATION_NOTES.md — engineering rules and scope definitions
4. docs/TEST_PLAN.md — validation checklist and test vector inventory
5. docs/modelsim_transcript.log — execution proof
6. docs/PINOUT_REQUIRED.md — pinout requirement warning

11.2 Immediate action checklist

1. Verify pin assignments: import official Terasic DE2-115 pin assignments into quartus/int8_transformer_de2_115.qsf (docs/PINOUT_REQUIRED.md).
2. FPGA synthesis: open the .qpf in Quartus Prime Lite → Start Compilation (Ctrl+L).
3. Timing analysis: extract Fmax and worst-case setup slack from the .sta.rpt.
4. Resource utilization: record LE, register, DSP, and M9K usage from the .fit.rpt.
5. Board programming: connect DE2-115 via USB-Blaster → program the .sof.
6. UART telemetry: open a serial monitor at 115,200 baud → capture output strings.

Compiled from KNOWLEDGE_BASE.md, IMPLEMENTATION_NOTES.md, TEST_PLAN.md, modelsim_transcript.log, and PINOUT_REQUIRED.md. All cycle counts and error metrics are ModelSim / Python simulation results as of the 2026-09-06 run; Quartus and physical-hardware figures are pending (§8).